

RecyPrint Software Architecture

Intelligent Waste-to-Filament Backend Systems

IOT · DATABASE · REAL-TIME MONITORING

Software Team Core Roles



Bridge Engineering

Creating a secure connection between physical sensors and cloud infrastructure.



Cloud Orchestration

Managing real-time data persistence and remote process control loops.

- To provide low-latency bidirectional data communication.
- To develop a user-friendly, API-supported dashboard.
- To establish an offline-first data architecture resilient to internet outages.
- Integration of cloud database (Neon) and local caching (SQLite).

Development Stack



Python & Flask

Core backend API and
middleware bridge
scripting.



SQL Ecosystem

Neon PostgreSQL (Cloud) &
SQLite (Local Fallback).



MQTT

Low-latency telemetry via
EMQX Public Broker.

| Process Workflow

- ✓ **Data Extraction:** Raw text logs (parsed via Regex) or structured JSON telemetry fetched via USB serial.
 - ✓ **Transmission:** Bidirectional Pub-Sub MQTT protocol relays real-time telemetry over a public cloud broker.
 - ✓ **Persistence:** Hybrid routing to Neon PostgreSQL (Cloud) or SQLite (Local Cache) to prevent data loss during outages.
 - ✓ **Visualization:** Direct MQTT-over-WebSockets for real-time dashboard updates, backed by Flask APIs for historical logs and CSV exports.
-

Communication Layer: MQTT

Compared to traditional poll-based HTTP, a lightweight, event-driven publish-subscribe model (MQTT) is selected for high-frequency IoT telemetry



Low-latency

Bidirectional, event-driven communication enabling sub-second response times for process controls.



Topic Architecture:

Data is isolated under structured topics (sensor_data, control, alerts) to optimize bandwidth and safety.



Middleware

Local Python bridge processes raw serial data and runs edge-level closed-loop control loops, minimizing cloud overhead.

| Real-Time Data Binding

The system binds incoming JSON payloads directly to UI components without page refreshes.

```
{  
  "temperature": 250.0,  
  "speed": 250,  
  "diameter": 1.75,  
  "status": "Extruding"  
}
```

-  Dynamic Chart.js updates every second.
-  Synchronous state management across clients.
-  Zero-lag physical command execution.

Backend API Architecture

```
@app.route('/api/logs')
@login_required
def get_logs():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            'SELECT temperature, speed, diameter, timestamp FROM sensor_data ORDER BY
id DESC LIMIT 100'
        )
        rows = cursor.fetchall()
        cursor.close()
        conn.close()
        # ... JSON serialization and response
```

RESTful Micro-services

- **/api/logs**: Serves historical telemetry data for trend analysis.
- **/export/csv**: Generates downloadable Excel/CSV reports for engineering analysis.
- **/login & /logout**: Manages secure, session-based user authentication.
- **Control Operations (Start/Stop)**: Triggered instantly via **asynchronous MQTT protocols** rather than HTTP REST, ensuring zero-lag physical execution.

| High-Frequency Telemetry

1 Hz

1 Packet / Second

Why 1 Hz Frequency?

To capture instantaneous changes in data such as temperature and engine speed.

To ensure that graphs on the interface appear smooth and uninterrupted (real-time) to users.

To enable the software to intervene immediately in emergencies (e.g., overheating).

| Cloud SQL Strategy

Neon PostgreSQL

The primary cloud engine provides global accessibility and scalable relational storage for massive time-series datasets.

Automated vertical scaling.

Secure environment variable authentication.

SQLite

A local database that temporarily writes data and only comes into play when the internet connection is lost.

| Write Optimization

Intelligent Write Throttling

To prevent database bloat, the software uses a selective commit strategy:

-  **Standard Mode: Commits 1 entry every 5 seconds. Zero-latency data handling ensures that all system alerts are prioritized for immediate database commit.**
-  **Critical state changes (e.g., temperature spikes, emergency stops) bypass the sampling filter for real-time consistency.**

| Offline Mode Caching

SQLite Resilience Layer

Interruption Detection: The backend immediately detects that the Neon PostgreSQL server is unreachable (socket error).

Fallback: Data write operations are automatically redirected to the SQLite file on the local disk.

Local Operation: Even without internet access, the device continues to generate data, and the software continues to store this data locally.

When internet connectivity fails, the Python bridge intercepts socket errors and reroutes telemetry to a local database file.

Zero configuration overhead.

Hardware timestamp preservation.

Atomic commit integrity.

| Background Sync Flow



| Backend Data Logic

Algorithmic Smoothing

Software-level EMA (Exponential Moving Average) eliminates high-frequency sensor noise before visualization.



Data Correction and Filtering

The software is responsible for cleaning electrical noise from incoming sensor data.



EMA (Exponential Moving Average) Algorithm: Data is smoothed programmatically to prevent sudden spikes.



Control Loop Analysis: Logging of system parameters to ensure PID accuracy and hardware thermal stability.

Software Guardrails



Critical Halt

Instant motor kill command
if temp > 270°C.



Auth Screen

Flask Session protection for
Dashboard access.



Heartbeat

Socket timeout alerts for
stale telemetry.

Historical Data Export



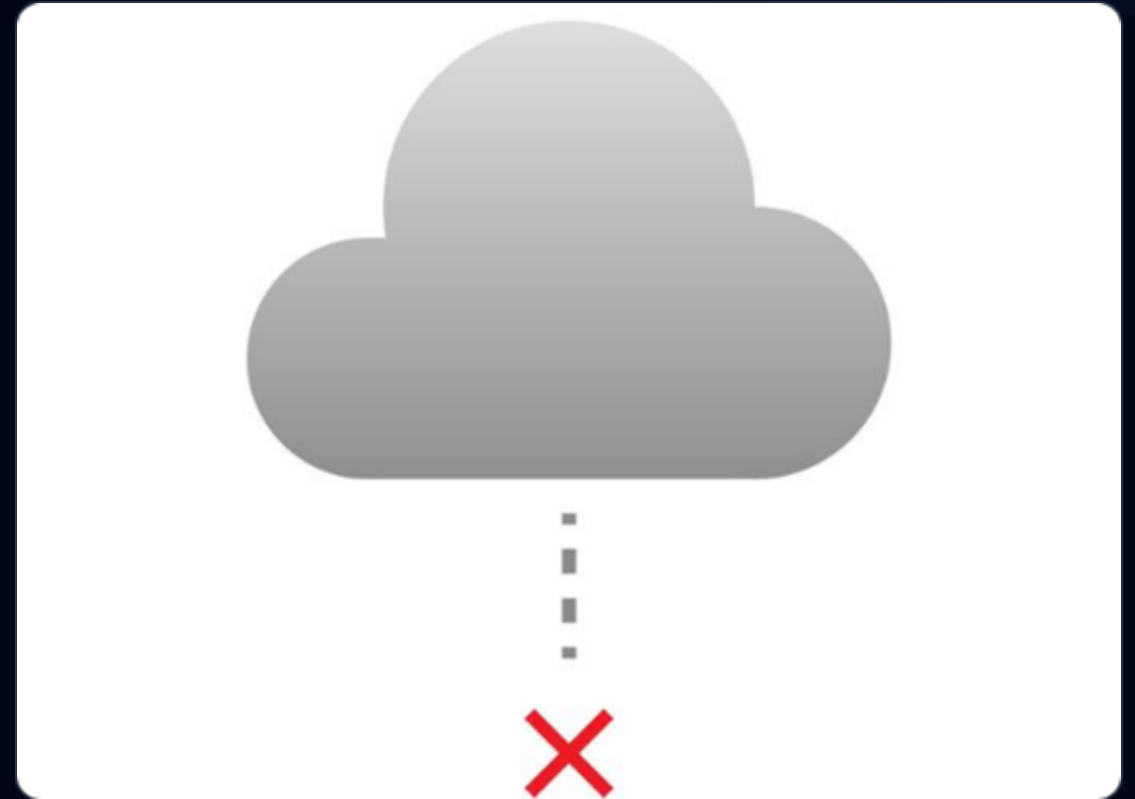
The Flask API provides a dedicated endpoint (/export/csv) for querying records in the database.



Users can download temperature and engine status history for specific date ranges in CSV format with a single click.



This data is used for system performance evaluations (e.g., temperature variation analyses).



| Operator Dashboard

The backend and API architecture provides an interface that allows the user to easily manage the system.

- Live data graphs (using Chart.js, etc.).
- System Start/Stop buttons (API triggers).
- Active connection status indicators (MQTT & Database status).

Data Flow Direction:

Hardware → MQTT → Flask Backend → API → Web Dashboard

Development Evolution



Software Excellence



Version Control

Modular repositories for
Bridge and Backend code.



AI Augmented

GitHub Copilot optimized
SQL and API logic.



Modular Code

Decoupled database drivers
for high maintainability.

Topology Summary

Sub-system	Standard / Tool	Frequency / Sync
HMI Dashboard	HTML5 / Chart.js	Real-time (1s Update)
Telemetry	MQTT over WSS	1 Hz Packet Rate
Cloud DB	Neon PostgreSQL	5s Buffered Writes
Local Backup	SQLite	Instant (Offline Mode)

Software System Ready

The RecyPrint software infrastructure is production-validated,
resilient to network outages, and ready for deployment.

 RecyPrint Core Repository

End-to-End IoT Software Delivery
